



Universidad Adolfo Ibáñez

Ingeniería en Computer Science

Informe Laboratorio 3

Arquitectura de Computadores

INTEGRANTES

- Martín Soto
- David Almonacid
- Enrique Larraín

Profesor: Cristian Astudillo

Sección: Sección 1

Fecha: 10 de junio de 2026

Resumen

En este informe se evalúan tres técnicas de optimización de código, predicción de saltos, padding y multihilos. Se uso la herramienta Perf para evaluar el efecto de estas técnicas en distintos codigos. En primer lugar se observo que para la predicción de saltos aleatorios, la aplicación de código branchless provoco una mejora en los branch misses de desde 1.30 % a 0.67 %. En segundo lugar, se midió el costo del falso compartido en sistemas multiprocesador. Al implementar estrategias de alineación de memoria (padding a 64 bytes), se disminuyó el tráfico de coherencia y se eliminó un 99.5 % de los fallos de lectura en la caché L1. Finalmente, se compararon tres estrategias de reducción paralela. Los resultados evidencian que las operaciones atómicas generan alta contención, mientras que la privatización de variables y la cláusula reduction de OpenMP minimizan este problema, logrando reducciones superiores al 99 % en ciclos consumidos y mejorando sustancialmente la escalabilidad.

1. Introducción

Los procesadores actuales incorporan numerosas técnicas para aumentar el rendimiento. Entre ellas destacan los mecanismos de predicción de saltos, que permiten ejecutar instrucciones de manera especulativa antes de conocer el resultado de una rama condicional, y los protocolos de coherencia de caché, encargados de mantener una visión consistente de la memoria entre múltiples núcleos.

Cuando un predictor de saltos falla, el procesador debe descartar instrucciones especulativas y reiniciar la ejecución desde la dirección correcta. Esto genera una penalización significativa en ciclos de reloj.

Por otra parte, en sistemas multiprocesador, múltiples núcleos pueden acceder simultáneamente a regiones cercanas de memoria. Si distintas variables ocupan una misma línea de caché, pueden producirse invalidaciones innecesarias conocidas como *false sharing*, reduciendo considerablemente el rendimiento.

Este laboratorio estudia experimentalmente ambos fenómenos mediante herramientas de profiling y análisis de eventos hardware.

2. Metodología

Entorno Experimental

Los experimentos fueron realizados en una computadora portátil Acer Swift SF316-51 ejecutando Linux Debian 13.

Procesador

El sistema cuenta con un procesador Intel Core i5-11300H de cuatro núcleos físicos y ocho hilos de ejecución, con una frecuencia base de 3.10 GHz.

La jerarquía de memoria caché del procesador corresponde a:

- Caché L1 de datos (L1d): 192 KiB (4 instancias)
- Caché L1 de instrucciones (L1i): 128 KiB (4 instancias)
- Caché L2: 5 MiB (4 instancias)
- Caché L3 compartida: 8 MiB

El equipo dispone de 8 GB de memoria LPDDR4 funcionando a una velocidad de 4267 MT/s.

2.1. Ejercicio 1: Predicción de Saltos

El objetivo de este experimento fue analizar cómo la predictibilidad de las ramas afecta el rendimiento del procesador.

Se utilizó un programa que recorre un vector de 100 millones de elementos y ejecuta una condición `if(val >threshold)`. Se evaluaron dos escenarios:

1. Datos ordenados, generando un patrón altamente predecible.
2. Datos mezclados aleatoriamente, generando un patrón impredecible.

Con el fin de evitar que el compilador eliminara o modificara el comportamiento de las ramas mediante optimizaciones agresivas, el programa fue compilado sin optimizaciones:

```
1 g++ -O0 -g branch_prediction.cpp -o branch_pred
```

Posteriormente se registraron métricas relacionadas con el flujo de control utilizando:

```
1 perf stat -e branch-misses,branches,  
2 cycles,instructions ./branch_pred
```

A partir de estos resultados se calculó la tasa de fallos de predicción y el CPI (Cycles Per Instruction).

2.2. Ejercicio 2: Falso Compartido

El segundo experimento tuvo como objetivo identificar el impacto del falso compartido sobre el rendimiento de aplicaciones paralelas.

Se utilizó un programa OpenMP en el cual dos hilos incrementan contadores distintos almacenados dentro de una misma estructura. Debido a que ambos contadores ocupan la misma línea de caché, se producen invalidaciones continuas provocadas por el protocolo de coherencia.

La compilación se realizó mediante:

```
1 g++ -O2 -fopenmp false_sharing.cpp
2 -o false_sharing
```

Para medir el comportamiento de la jerarquía de memoria se utilizó:

```
1 perf stat -e cache-misses ,
2 L1-dcache-load-misses ,
3 cycles ./false_sharing
```

Posteriormente se implementó una versión optimizada utilizando alineación explícita mediante `alignas(64)`, separando ambos contadores en distintas líneas de caché.

Las métricas obtenidas permitieron comparar el efecto del falso compartido antes y después de la optimización.

2.3. Ejercicio 3: Reducción Paralela

El tercer experimento estudió distintas estrategias para realizar una suma paralela de un vector de 100 millones de elementos.

Se compararon tres implementaciones:

1. Uso de operaciones atómicas (`atomic`).
2. Uso de la cláusula `reduction` de OpenMP.
3. Implementación manual mediante variables privadas por hilo.

La compilación se realizó utilizando:

```
1 g++ -O2 -fopenmp parallel_reduction.cpp
2 -o parallel_reduction
```

Las mediciones se efectuaron mediante:

```

1 perf stat -e cache-misses,
2 LLC-load-misses,
3 cycles ./parallel_reduction

```

Los resultados permitieron evaluar el impacto de la sincronización y del tráfico de coherencia sobre la escalabilidad de cada implementación.

3. Resultados y análisis

3.1. Ejercicio 1: Medición del Costo de Fallos de Predicción

Objetivo

Medir el costo de los fallos de predicción de saltos y analizar cómo la predictibilidad de las ramas afecta el rendimiento de un programa, utilizando métricas obtenidas mediante `perf` y comparando una implementación tradicional con una versión branchless.

Tiempo de ejecución

Cuadro 1: Tiempo de ejecución para los distintos patrones de ramas

Patrón de datos	Original (s)	Branchless (s)
Ordenado (predecible)	1.29623	1.34219
Aleatorio (impredecible)	2.38074	1.37636

Métricas de predicción de saltos

Cuadro 2: Eventos de predicción obtenidos mediante `perf`

Caso	Branch Misses	Branches	Tasa de Fallos (%)	IPC
Ordenado (Original)	74 465	2 461 355 279	0.0030	2.84
Ordenado (Branchless)	49 200	2 323 824 233	0.0021	2.90
Aleatorio (Original)	102 458 432	7 900 344 233	1.30	1.39
Aleatorio (Branchless)	52 503 141	7 790 856 260	0.67	1.48

Comparación de la optimización

Cuadro 3: Impacto de la optimización branchless

Métrica	Ordenado	Aleatorio
Reducción de Branch Misses	33.9 %	48.8 %
Mejora en tiempo de ejecución	-3.5 %	42.2 %
Incremento de IPC	2.1 %	6.5 %

Análisis y Discusión

los resultados muestran una fuerte relación entre la predictibilidad de las ramas y el rendimiento del programa. Cuando los datos se encuentran ordenados, la condición evaluada dentro del ciclo presenta un comportamiento altamente predecible, permitiendo que el Branch Prediction alcance una tasa de error prácticamente nula. En este escenario se registraron únicamente 74 465 fallos de predicción sobre más de 2.4 mil millones de ramas ejecutadas, equivalente a una tasa aproximada de 0.003 %.

Debido a esta elevada precisión, la versión branchless no genera beneficios significativos. De hecho, el tiempo de ejecución aumenta levemente desde 1.29623 s hasta 1.34219 s. Esto indica que, cuando el Branch Prediction es capaz de anticipar correctamente el comportamiento de la rama, el costo adicional de las operaciones utilizadas para eliminar el salto puede superar cualquier mejora potencial.

La situación cambia de manera importante para el caso aleatorio. En este escenario la condición evaluada resulta difícil de predecir y el número de errores aumenta hasta 102 458 432 branch misses, alcanzando una tasa de fallos de 1.30 %. Como consecuencia, el tiempo de ejecución aumenta hasta 2.38074 s y el IPC disminuye a 1.39 instrucciones por ciclo.

La implementación branchless reduce los errores de predicción a 52 503 141 branch misses, disminuyendo la tasa de fallos a 0.67 %. Esta reducción cercana al 49 % permite disminuir el tiempo de ejecución a 1.37636 s, obteniendo una mejora aproximada del 42 %. Además, el IPC aumenta hasta 1.48, reflejando una utilización más eficiente del pipeline.

Estos resultados muestran una relación directa entre el número de fallos de predicción y el tiempo de ejecución. Cada error obliga al procesador a descartar instrucciones ejecutadas especulativamente y reiniciar parte del pipeline, aumentando la cantidad de ciclos necesarios para completar el programa. Esto explica por qué el caso aleatorio presenta simultáneamente una mayor tasa de branch misses y un tiempo de ejecución considerablemente superior.

Asimismo, los resultados confirman que las técnicas branchless constituyen una estrategia efectiva para reducir la dependencia del Branch Prediction cuando las ramas presentan patrones impredecibles. Sin embargo, cuando el patrón es altamente predecible, como ocurre

en el caso ordenado, la eliminación de la rama no aporta beneficios significativos e incluso puede introducir un pequeño costo adicional.

Finalmente, los resultados permiten comprender por qué los procesadores modernos dedican una cantidad considerable de recursos hardware al diseño de Branch Predictions cada vez más sofisticados. Incluso reducciones relativamente pequeñas en la tasa de error pueden evitar millones de vaciados de pipeline y traducirse en mejoras significativas de rendimiento.

3.2. Ejercicio 2: Análisis de Falso Compartido en Sistemas Multi-procesador

Objetivo

Analizar el impacto del falso compartido sobre el rendimiento de una aplicación paralela y evaluar la efectividad de una solución basada en padding y alineación de memoria para reducir el tráfico de coherencia entre núcleos.

Cuadro 4: Métricas de caché antes y después de corregir el falso compartido

Métrica	Con falso compartido	Con padding
Cache Misses	135 443	75 222
L1-dcache-load-misses	51 154 301	214 921
Cycles	7 721 532 345	4 710 089 147
Tiempo (s)	2.57758	2.53046

Cuadro 5: Reducción obtenida tras eliminar el falso compartido

Métrica	Reducción (%)
Cache Misses	44.5
L1-dcache-load-misses	99.58
Cycles	39.0
Tiempo de ejecución	1.83

Análisis y Discusión

Los resultados evidencian el efecto del falso compartido sobre el comportamiento de la jerarquía de memoria. En la implementación original, ambos hilos modifican variables distintas que se encuentran almacenadas dentro de una misma línea de caché. Aunque las variables son independientes desde el punto de vista lógico, cada escritura provoca invalidaciones continuas entre los núcleos debido al protocolo de coherencia.

Este fenómeno puede observarse especialmente en la métrica `L1-dcache-load-misses`. La versión original registró más de 51 millones de fallos en caché L1, mientras que la versión optimizada redujo esta cifra a aproximadamente 215 mil. Esto representa una disminución grande correspondiente a un 99.5 %, indicando que gran parte del tráfico observado en la versión original estaba asociado a invalidaciones provocadas por el falso compartido.

Asimismo, la cantidad total de ciclos consumidos disminuyó desde aproximadamente 7.72 mil millones hasta 4.71 mil millones de ciclos, equivalente a una reducción cercana al 39 %. Esta diferencia muestra que una fracción importante del tiempo de ejecución era utilizada esperando transferencias de líneas de caché entre núcleos.

La optimización consistió en separar físicamente ambos contadores mediante padding y alineación a 64 bytes, garantizando que cada variable se almacenara en una línea de caché distinta. De esta forma, cada núcleo pudo actualizar su contador sin interferir con el otro, reduciendo significativamente el tráfico de coherencia.

Aunque la mejora observada en el tiempo total de ejecución fue relativamente moderada (1.83 %), las métricas internas muestran una reducción muy significativa en los accesos fallidos a caché y en los ciclos consumidos. Esto demuestra que el falso compartido puede afectar severamente la eficiencia del subsistema de memoria incluso cuando el impacto sobre el tiempo total de ejecución parece pequeño.

En conjunto, los resultados confirman que la organización física de los datos en memoria es un aspecto fundamental en aplicaciones paralelas. Técnicas simples como el uso de padding y alineación explícita permiten reducir considerablemente el tráfico de coherencia y mejorar la eficiencia de programas multiprocesador.

3.3. Ejercicio 3: Comparación de Estrategias de Reducción Paralela

Objetivo

Comparar distintas estrategias de acumulación en programas paralelos, evaluando el impacto de la sincronización sobre el rendimiento. En particular, se analizaron tres enfoques: actualización mediante operaciones atómicas, privatización de variables y la cláusula `reduction` de OpenMP, utilizando métricas de rendimiento obtenidas con `perf`.

Cuadro 6: Comparación de estrategias de reducción paralela

Implementación	Tiempo (s)	Cache Misses	L1-dcache-load-misses	Cycles
Atomic	15.5213	22 358 616	188 061 146	178 313 391 650
Privatización	0.0525	20 872 320	42 849 242	1 462 831 560
Reduction (OpenMP)	0.0323	20 831 110	42 148 259	1 251 370 233

Cuadro 7: Mejora respecto de la implementación Atomic

Métrica	Privatización	Reduction
Reducción de Cycles	99.18 %	99.30 %
Reducción de L1 Misses	77.22 %	77.59 %
Mejora en tiempo de ejecución	99.66 %	99.79 %

3.4. Análisis y Discusión

Los resultados muestran diferencias muy significativas entre las estrategias de reducción evaluadas. La implementación basada en operaciones atómicas presentó el peor rendimiento, requiriendo 15.52 segundos para completar la suma. En contraste, las versiones basadas en privatización y en la cláusula `reduction` de OpenMP completaron la misma tarea en aproximadamente 0.05 y 0.03 segundos respectivamente.

La principal causa de esta diferencia es la contención generada por la variable compartida utilizada en la versión Atomic. Cada actualización requiere mecanismos de sincronización que impiden que múltiples hilos modifiquen simultáneamente la misma ubicación de memoria. Como consecuencia, el paralelismo efectivo disminuye considerablemente y los hilos pasan gran parte del tiempo esperando acceso a la variable compartida.

Este efecto también se refleja en la cantidad de ciclos consumidos. La implementación Atomic requirió más de 178 mil millones de ciclos, mientras que las versiones con privatización y reduction utilizaron aproximadamente 1.46 mil millones y 1.25 mil millones de ciclos respectivamente. Esto representa una reducción superior al 99 % en ambos casos.

Las métricas de caché muestran un comportamiento similar. Aunque el número total de cache misses presenta diferencias moderadas, los fallos de caché L1 disminuyen desde 188 millones hasta aproximadamente 42 millones en las versiones optimizadas. Esto indica una reducción importante del tráfico asociado a la variable compartida y una utilización más eficiente de la jerarquía de memoria.

Entre las dos soluciones optimizadas, la implementación mediante `reduction` obtuvo el mejor rendimiento general. Esto sugiere que OpenMP realiza optimizaciones internas eficientes para combinar los resultados parciales generados por cada hilo, minimizando el costo de sincronización.

En conjunto, los resultados demuestran que las operaciones atómicas pueden convertirse en un cuello de botella severo en aplicaciones paralelas cuando existe una alta frecuencia de actualizaciones sobre una variable compartida. La privatización de datos y las reducciones nativas de OpenMP permiten eliminar gran parte de esta contención, mejorando significativamente la escalabilidad y el rendimiento del programa .

4. Conclusiones

Los experimentos realizados ayudaron comprender de manera práctica cómo distintos aspectos de la arquitectura de computadores influyen directamente en el rendimiento de las aplicaciones. A través del análisis de predicción de saltos, coherencia de caché y sincronización en programas paralelos, fue posible observar que el desempeño de un programa no depende únicamente de la cantidad de operaciones ejecutadas, sino también de cómo interactúa con los mecanismos internos del procesador.

En el primer ejercicio se comprobó que los patrones de ejecución predecibles permiten al Branch Prediction operar con una tasa de error extremadamente baja, mientras que los patrones aleatorios generan una cantidad significativamente mayor de fallos de predicción. Estos errores obligan al procesador a descartar trabajo realizado especulativamente, incrementando el número de ciclos consumidos y el tiempo de ejecución.

En el segundo ejercicio se observó el impacto del falso compartido en sistemas multiprocesador. Aunque los hilos trabajaban sobre variables distintas, el hecho de que estas compartieran una misma línea de caché provocó invalidaciones constantes y un aumento considerable de los fallos de caché. La utilización de padding y alineación de memoria permitió reducir significativamente este problema, demostrando la importancia de considerar la organización física de los datos además de su estructura lógica.

Finalmente, el tercer ejercicio mostró que las operaciones atómicas pueden convertirse en un importante cuello de botella cuando múltiples hilos compiten por actualizar una misma variable compartida. Las técnicas de privatización y la cláusula reduction de OpenMP eliminaron gran parte de esta contención, obteniendo mejoras de rendimiento muy significativas y evidenciando la importancia de minimizar la sincronización innecesaria en aplicaciones paralelas.

Donde La aplicación de estos principios permite aprovechar de mejor manera los recursos hardware disponibles y desarrollar programas paralelos más escalables y eficientes.

Codigo fuente

Los codigos y logs pueden ser encontrados aqui

Aclaración: El uso de inteligencia artificial en este informe se limitó exclusivamente a fines estéticos y a la mejora de la redacción del documento.